

TEKNOFEST

**HAVACILIK, UZAY VE TEKNOLOJİ
FESTİVALİ**

**ULAŞIMDA YAPAY ZEKA YARIŞMASI
KRİTİK TASARIM RAPORU**

Ai Lab-SEMRÜK

315851

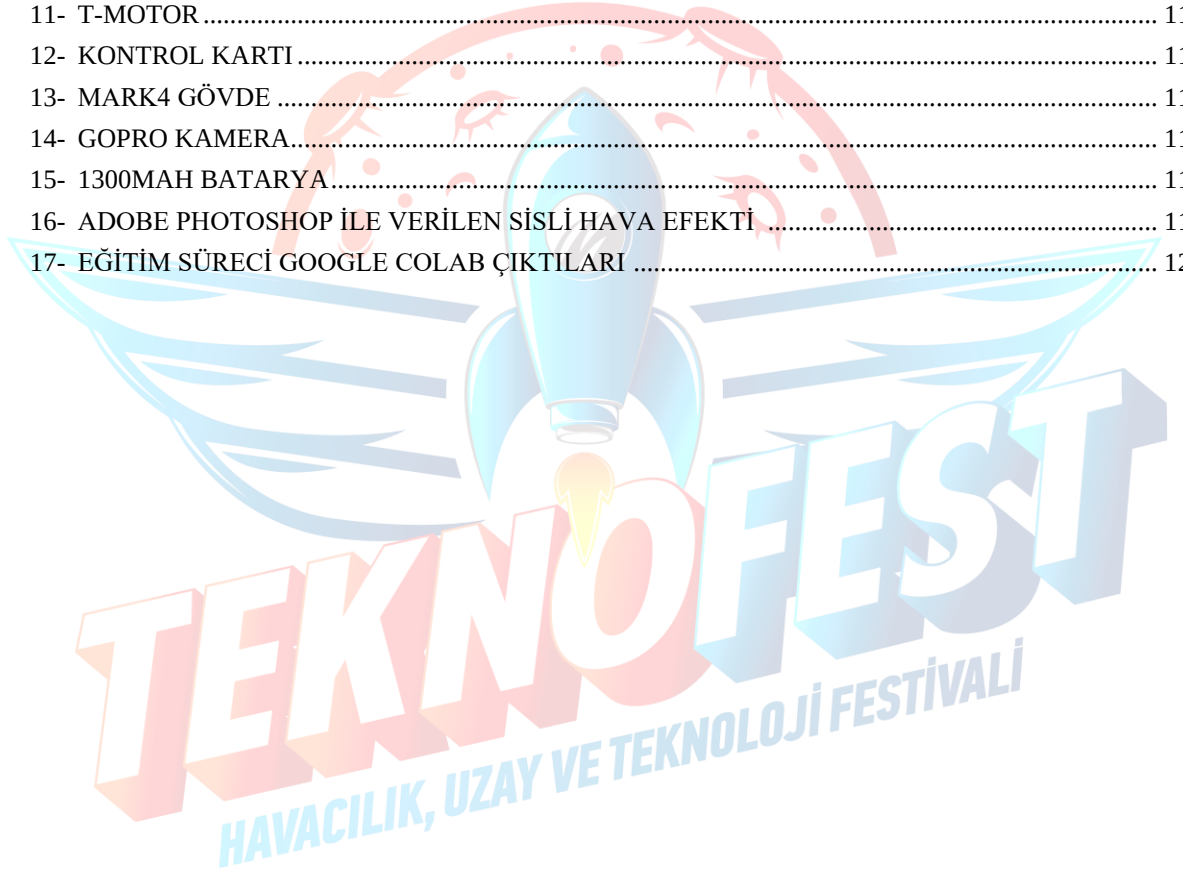
İÇİNDEKİLER

İÇİNDEKİLER	2
ŞEKİL LİSTESİ.....	3
KISALTMALAR	4
1. TAKIM ŞEMASI	5
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ	6
3. ALGORİTMALAR VE SİSTEM MİMARİSİ.....	6
3.1. Veri Setleri.....	6
3.2. Algoritmalar	6
3.2.1. YOLO Algoritması	6
3.2.2. MASK R-CNN Algoritması	7
3.3. Sistem Mimarisi.....	9
3.3.1. Etiket Büyüklük Kontrolü (Label Size Control):	9
3.3.2. İki Algoritmayı Birleştirme (Combination of Two Algorithms):.....	10
4. ÖZGÜNLÜK	10
4.1. Etiket Büyüklük Kontrolü (Label Size Control).....	10
4.2. İki Algoritmayı Birleştirme (Combination of Two Algorithms).....	10
4.3. UAP ve UAİ Veri Seti Genişletme	10
4.4. Ürettiğimiz Drone ile Veri Seti Oluşturma	10
4.5. Fotoğraf Filtreleme	11
4.6. Deneme Eğitim Sonuçlarını Kullanma	11
5. SONUÇLAR VE İNCELEME.....	11
6. KAYNAKÇA.....	13



ŞEKİL LİSTESİ

1- TAKIM ŞEMASI.....	5
2- LABELIMG ETİKETLEME İŞLEMİ.....	6
3- NON-MAXİMUM SUPRESSION ALGORİTMASI.....	7
4- YOLO MODELLERİNİN KARŞILAŞTIRILMA GRAFİĞİ	7
5- GÖRME SINIFLANDIRMA ÇEŞİTLERİ	8
6- BACKBONE MİMARİSİ.....	9
7- LABEL SİZE CONTROL ÖNCESİ.....	10
8- LABEL SİZE CONTROL SONRASI	10
9- İKİ ALGORİTMA ÇIKTISI.....	10
10- TEK ALGORİTMA ÇIKTISI.....	10
11- T-MOTOR	11
12- KONTROL KARTI.....	11
13- MARK4 GÖVDE	11
14- GOPRO KAMERA.....	11
15- 1300MAH BATARYA.....	11
16- ADOBE PHOTOSHOP İLE VERİLEN SİSLİ HAVA EFEKTİ	11
17- EĞİTİM SÜRECİ GOOGLE COLAB ÇIKTILARI	12



KISALTMALAR

K: Ultra High Definition (UHD)

FPS: Frame Per Second

YOLO: You Only Look Once

R-CNN: Region Based Convolutional Networks

UAP: Uçan Araba Park Alanı

UAI: Uçan Ambulans İniş Alanı

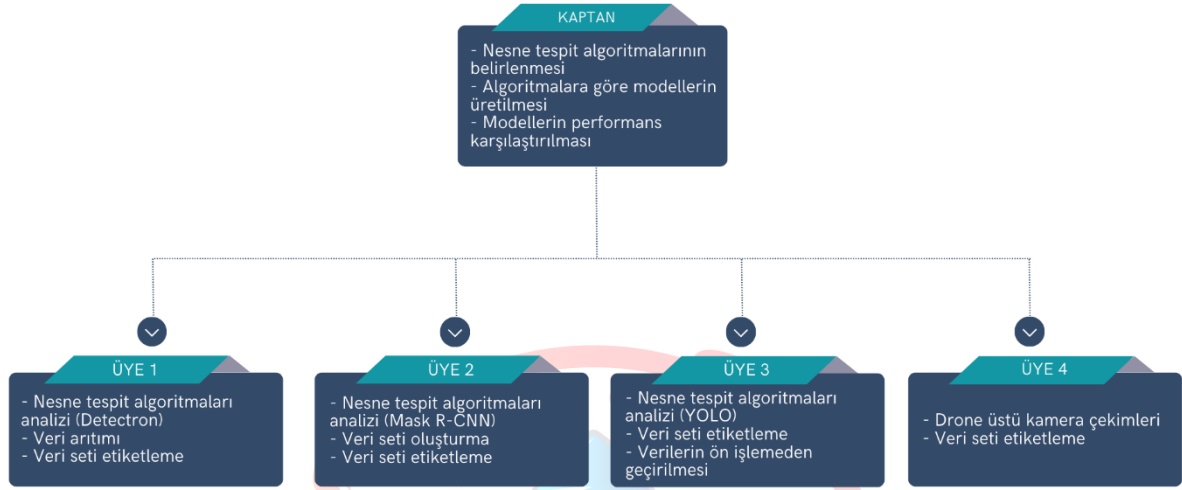
GB: Gigabayt

RAM: Random Access Memory

GPU: Graphics Processing Unit



1. TAKIM ŞEMASI



Şekil 1.1 Takım Şeması

Ai Lab-SEMRÜK takımı bir danışman, bir takım kaptanı ve 4 üye olmak üzere toplam 6 kişiden oluşmaktadır.

Sait Ali Uymaz (Danışman): Konya Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü öğretim üyesidir. Takıma hangi algoritmaların yararlı olabileceğinden bahseder ve araştırma görevleri verir.

Kaptan: Konya Teknik Üniversitesi Bilgisayar Mühendisliği öğrencisidir. Yapay zeka ve otonom sistemlere ilgisi bulunmaktadır. Takımın zaman-görev çizelgesini oluşturur ve takım üyelerine ilgi alanına en yakın görevleri verir.

Üye 1: Konya Teknik Üniversitesi Bilgisayar Mühendisliği öğrencisidir. Yapay zeka ve algoritmalarla ilgisi bulunmaktadır. Takımdaki görevi veri seti etiketi ve veri artırımı yapmak, eğitim yapılabilecek platformları araştırmaktır.

Üye 2: Konya Teknik Üniversitesi Bilgisayar Mühendisliği öğrencisidir. Yapay zeka ve otonom sistemlere ilgisi bulunmaktadır. Takımdaki görevi veri seti oluşturmak ve veri setlerini etiketlemektir.

Üye 3: Konya Teknik Üniversitesi Bilgisayar Mühendisliği öğrencisidir. Yapay zeka ve mobil uygulama geliştiriciliğine ilgisi vardır. Takımdaki görevi veri seti etiketi yapmak, verileri ön işlemeden geçirmek ve sosyal medya organizasyonu ile ilgilenmektir.

Üye 4: Selçuk Üniversitesi Elektrik Elektronik Mühendisliği öğrencisidir. Dronelar ve drone donanımına ilgisi vardır. TEKNOFEST Üniversiteler Arası Drone Yarışması'nda Türkiye derecesi bulunmaktadır. Takımdaki görevi veri seti etiketlemek ve kendi yaptığı droneuyla veri setimizi geliştirmektir.

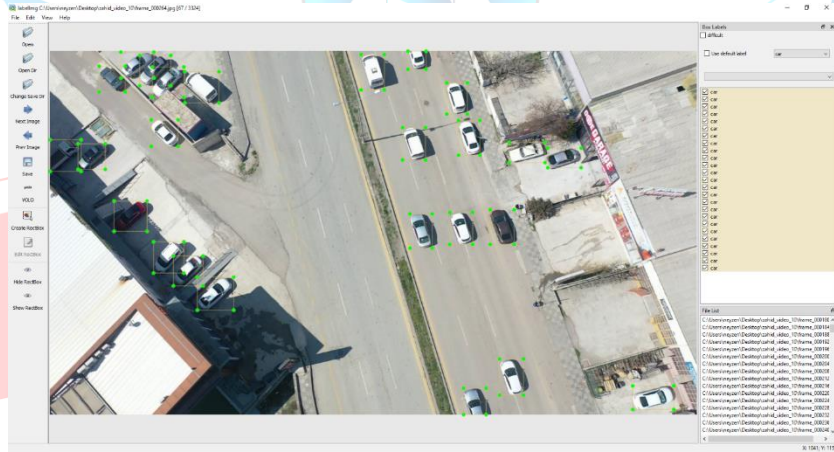
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ

Ai Lab-SEMRÜK takımı olarak ön tasarım raporunda belirtilen proje takvimine göre çalışmalarımızı sürdürmeye özen gösterdik. Veri seti oluşturma işleminin devamı için T3 Vakfı tarafından yarışmaya katılan takımlarla paylaşılan örnek video ve geçmiş senelerin yarışma videoları framelerine ayrılmıştır. Yine veri seti için takım üyemiz İbrahim Alper Aydın tarafından geliştirilen drone ile eksik görülen insan, kamyon ve tır veri seti güçlendirilmiştir. Veri setinin çeşitlendirilmesi amacıyla seçilen bazı framerler üzerine Adobe Photoshop CS6 ile sis, yağmur ve kar efektleri verilmiştir.[1] Ön tasarım raporunda ön araştırmaları yapılan YOLOv5, Faster R-CNN, Mask R-CNN ve Detectron algoritmaları üzerinde bir dizi deney yapılmıştır. Sistem gereksinimleri düşük ve kullanımı kolay olan YOLOv5m6 nesne tespit algoritması eğitim süreci için seçilmiştir.[2] Yapay zeka eğitimi için takım üyelerinin ve KTÜN laboratuvarındaki bilgisayarların donanımın yetersiz olması sebebiyle Google Colab Pro üzerinden eğitim yapılmasına karar verilmiştir. Google Colab (Google Colaboratory), yapay zeka ve derin öğrenme projeleri üzerinde çalışanlar için etkileşimli, tamamen bulut tabanlı, kullanımı kolay ve ortak çalışmaya dayalı bir programlama ortamıdır.[3] Yaptığımız eğitimlerde Google Colab Pro tarafından sağlanan Tesla P100 16 GB GPU ve 25 GB RAM donanımları kullanılmıştır.

3. ALGORİTMALAR VE SİSTEM MİMARİSİ

3.1. Veri Setleri

Eğitim süresince kullanılan veri setleri (seçilen bazı framerler üzerinde karlı, yağmurlu ve sisli görüntü efekti verilen) drone üstü kamera çekimleri, T3 vakfı tarafından sağlanan yarışma örnek videosu ve TEKNOFEST tarafından paylaşılan geçmiş senelerin yarışma videoları olarak 3 sınıfa ayrılabilir. Takımımız, bu veri setlerini framerlerine ayırmış ve her takım üyesine eşit sayıda frame düşecek şekilde paylaşmıştır. Paylaşılan bu veri setleri üzerindeki etiketleme işlemi GitHub üzerinden açık kaynak kodlu paylaşılan LabelImg yazılımı ile devam etmektedir. [4]



Şekil 3.1 LabelImg Etiketleme İşlemi

Yapay zeka test eğitimleri belli periyotlarda yapılmakta olup, etiketleme işleminin yapay zeka eğitimi üzerindeki olumlu-olumsuz etkileri tespit edilmektedir. Çalışmalarımız bu yöntemle devam etmektedir. Süreç sonunda yapay zeka performansının optimum düzeye çıkarılması hedeflenmektedir. Mevcut durumda eğitimlerimiz 10.000 frame ile yapılmaktadır. Online sınav tarihine kadar bu sayının 20.000'e çıkarılması hedeflenmektedir.

3.2. Algoritmalar

3.2.1. YOLO Algoritması

Takımımız olarak görüntü işlemede YOLO (You Only Look Once) algoritmasını tercih ettik. Bu algoritmayı tercih etmemizdeki en büyük sebebimiz ise isminin açılımından da anlaşılacağı üzere nesne tespitini oldukça hızlı ve tek seferde yapmaktadır. Diğer görüntü işleme algoritmalarına kıyasla daha

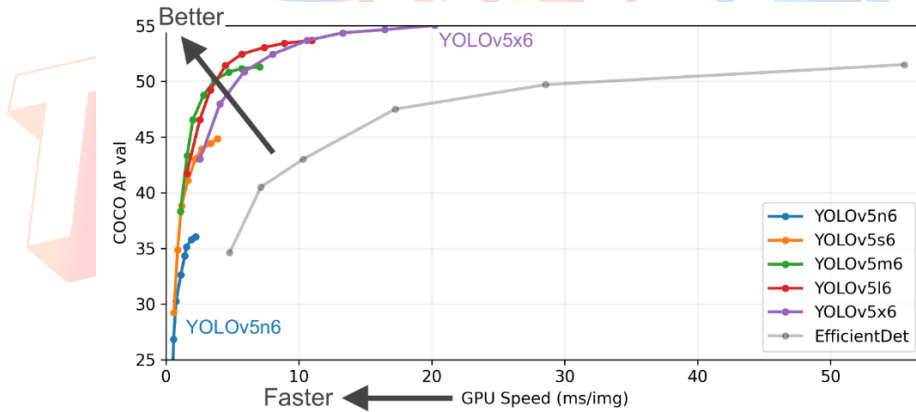
hızlıdır. YOLO algoritmasının diğer algoritmalarından hızlı olmasının sebebi resmin tamamını parçalara ayırmadan nöral sinir ağlarından geçirebilmesidir.

YOLO kendisine verilen resmi $N \times N$ boyutlarında ızgaralara böler. Bu ızgaralar 5×5 , 9×9 veya 17×17 boyutlarında olabilir. Her ızgara birbirinden bağımsız şekilde çalışır. Bir ızgara kendi alanlarında nesne var olup olmadığını ve eğer nesne var ise merkez noktası kendi alanında mı diye düşünür. Eğer nesnenin merkez noktasının kendi alanlarında olduğuna karar verirse nesne genişliğini ve yüksekliğini bulup o sınırlar içine nesneyi esas alan bir kutucuk (bounding box) çizmelidir. Bu işlem sırasında ızgaralar birbirinden bağımsız bounding box oluşturur. Bu durumda da ekranda gereksiz birçok bounding box gözükür. Oluşan bu sorun ise Non-Maximum Supression algoritması ile ortadan kaldırılır. Non-Maximum Supression algoritması oluşturulan bounding boxların güven skorlarına bakarak en yüksek güven skoruna sahip bounding box ekrana çizer. Böylece gereksiz kutucuk oluşumunun önüne geçilir.



Şekil 3.2 Non-Maximum Supression Algoritması

YOLO algoritması sürekli bahsettiğimiz üzere diğer algoritmaları kıyasla daha hızlı olmasının yanında, belirgin farkları olan nesnelerin eğitiminde doğruluk oranı olarak da diğer algoritmalarından daha iyi bir sonuç verir. Bunun yanı sıra YOLO algoritması Mask-RCNN, Detectron gibi maskeleme ile nesne algılaması yapan algoritmalarla kıyasla çok daha az yer kaplar.



Şekil 3.3 YOLO Modellerinin Karşılaştırılma Grafiği

Rahat, esnek ve daha birçok kolaylığı ile yapay zeka eğitimi aşamasında tercihimiz YOLOv5 algoritması olmuştur. Bu algoritmanın da YOLOv5m6 modeli hazırlamış olduğumuz veri seti ile beklenilenin üstünde bir doğrulukta çıktı vermektedir.

3.2.2. MASK R-CNN Algoritması

Bu sistem bilgisayarlı görü alanındaki instance segmentation (örnek segmentasyon) sorununu çözmeyi hedefleyen derin bir sinir ağıdır.[5] Mask R-CNN bir görseldeki nesneleri ayırabilir. Instance Segmentation, piksel düzeyindeki nesnelerin ana hatlarını tamamlayıp çıktı oluşturmaktır.

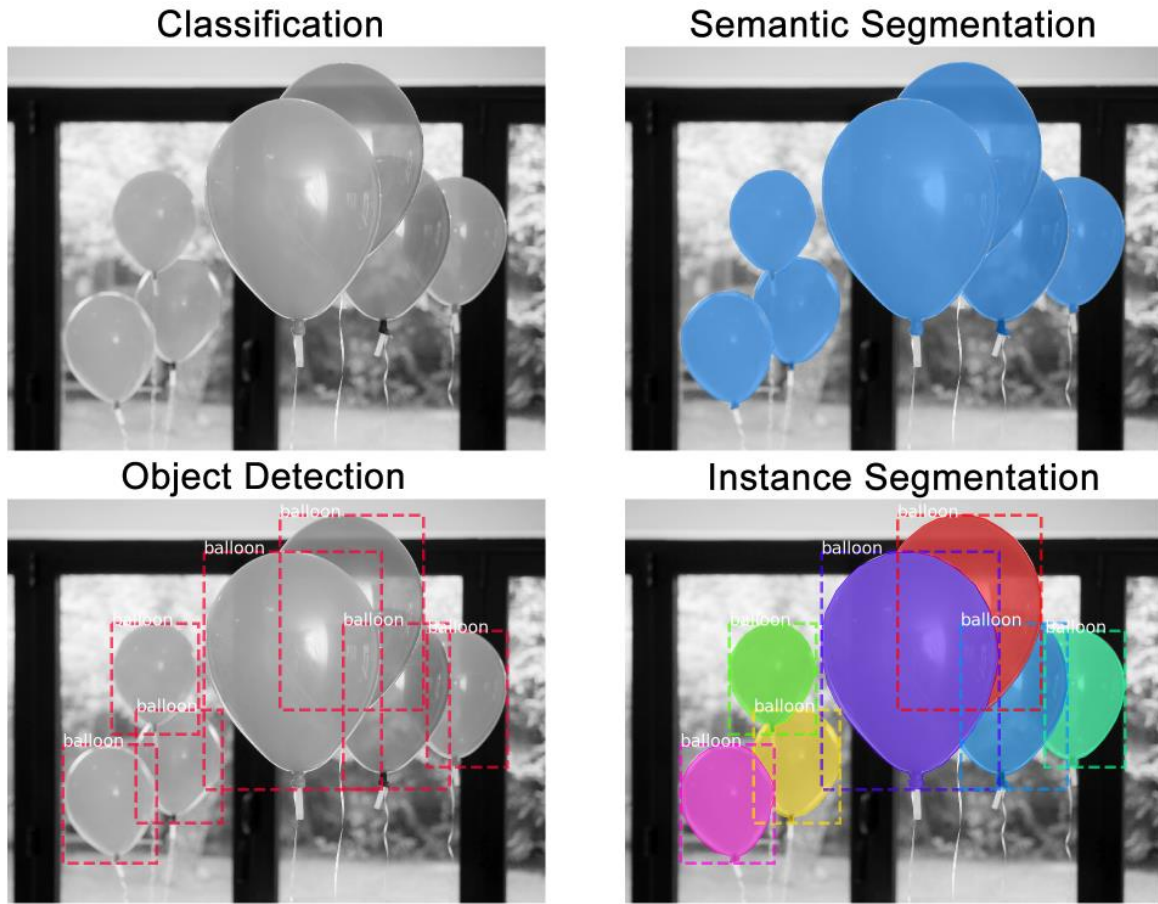
Benzer bilgisayarlı görü görevlerine kıyasla (örneğin: Object Detection) mümkün olan en zor görme görevlerinden biridir. Aşağıdaki örnekte de görüleceği üzere;

Classification (Sınıflandırma): Bu resimde bir balon olduğunu algılar.

Semantik Segmentation (Anlamsal Segmentasyon): Maskelenen alanların tamamı balon piksellerine aittir.

Object Detection (Nesne Tespiti): Görüntüdeki bu konumlarda balon var. Kesişen balonlar da algılamaya dahil edilir.

Instance Segmentation (Örnek Segmentasyon): Bu konumlarda balonlar var ve bunlar hepsine ait piksellerdir.



HAVACI

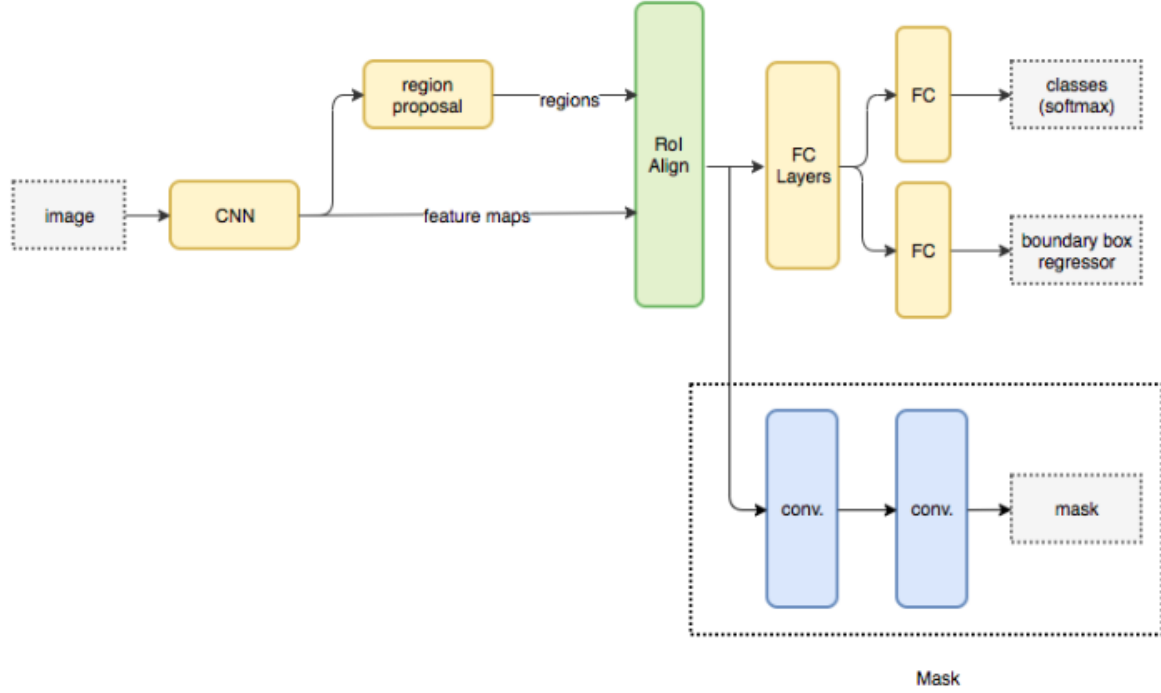
Şekil 3.4 Görme Sınıflandırma Çeşitleri

Mask R-CNN iki aşamalı bir sisteme sahiptir. İlk aşama görüntüyü tarar ve nesne tespiti için tahminlerde bulunur. Her konum için de tahmin bulunmaz. Nesne içerebilecek konumlarda tahminde bulunur. İkinci aşama ise bu önerileri değerlendirir, bounding box içine alır ve maskeler. Bu iki aşama da Backbone(Omurga) mimarisine bağlıdır.

Backbone(Omurga) Yapısı: Bu mimari resim içinde bulunan bounding boxlarının konumunu ve nesnelerin türlerini tespit eder. Backbone mimarisinin ilk katmanları düşük seviyeli özellikler olan kenar ve köşeleri tespit ederken ikinci katman daha üst seviyeli özellik olan nesne ayırımı yapar ve çıktı verir. Ayrıca Backbone mimarisi FPN tarzı bir derin sinir ağıdır. 3 ayrı yoldan oluşur:

- 1) Bottom-Up Pathway (Aşağıdan Yukarıya Yol): Bottom-Up Pathway hem görüntülerden özellikler çıkaran, genellikle ResNet veya VGG olmak üzere herhangi bir ConvNet'e benzer.
- 2) Top-Bottom Pathway (Yukarıdan Aşağıya Yol): Top-Bottom Pathway boyut olarak yukarıdan aşağıya piramite benzer bir özellik piramidi oluşturur.

- 3) Lateral Connections (Yanal Yol): Lateral Connections yukarıdaki iki yol arasında evrişim ve ekleme işlemleri yapar.



Şekil 3.5 Backbone Mimarisi

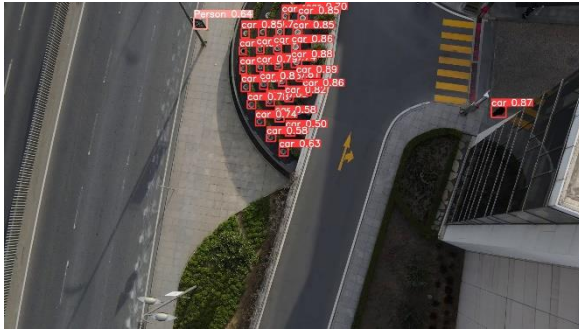
Ancak Mask R-CNN her ne kadar zoru başarıp güzel bir sonuç veriyor olsa da sistem yavaşlığı ve üst seviye bir donanım gereksinimi ile bizleri kısıtlamaktadır. Bu sebeplerden dolayı takım olarak görüntü işleme algoritması olarak tercihimiz YOLOv5 algoritması olmuştur.

3.3. Sistem Mimarisi

3.3.1. Etiket Büyüklük Kontrolü (Label Size Control):

Takımımız tarafından özgün bir şekilde hazırlanan büyüklük kontrolü sağlayan bir algoritmadır. Bu algoritmanın çalışma mantığı şu şekildedir:

Eğer kullandığımız görüntü işleme algoritması insandan küçük bir araba bulduysa veya arabadan büyük bir insan bulduysa tasarladığımız bu algoritma devreye girmekte ve düzeltmelere başlamaktadır. Arka tarafta insandan küçük araba olamaz ve arabadan büyük insan olamaz gibi karar mekanizmaları çalışmaktadır. Label Size Control ile düzeltilmiş bir frame örneği Şekil 7 ve Şekil 8'de gösterilmiştir.



Şekil 3.6 Label Size Control Öncesi



Şekil 3.7 Label Size Control Sonrası

3.3.2. İki Algoritmayı Birleştirme (Combination of Two Algorithms):

Bu sistemde ise asıl hedeflenen sonuç YOLOv5l6 modelinin araçlar üzerindeki kalitesi ve YOLOv5m6 modelinin ise küçük nesneleri algılamadaki kararlılığını birleştirip ortaya daha kaliteli bir sonuç çıkarmaktır. Sonuç olarak YOLOv5m6 modelinden gelen veriler ile YOLOl6 modelinden gelen verilerin birleştirilip tekrardan bir Non-Maximum Supression işlemi uygulayarak en yüksek güven skoruna ait kutucuk çizdirilecektir. Algoritmanın örnek çıktısı Şekil 9 ve Şekil 10'da gösterilmiştir.



Şekil 3.8 İki Algoritma Çıktısı



Şekil 3.9 Tek Algoritma Çıktısı

4. ÖZGÜNLÜK

Nesne etiketlerine daha tutarlı karar verilmesi, veri setinin güçlendirilmesi ve eğitimin en doğru şekilde sonuç vermesi için aşağıda verilen yazılım ve uygulamalar tasarlanmıştır:

4.1. Etiket Büyüklük Kontrolü (Label Size Control)

Etiketlenen veri setlerinin piksel boyutlarına bakılarak etiketin hangi nesne olup olmayacağına karar veren bir algoritmadır. Örneğin etiketlenmiş nesne 15 x 15 boyutlarından büyük ise insan olamaz gibi kararlar vererek insan-araç ayrımının yapılması kolaylaştırılmıştır.

4.2. İki Algoritmayı Birleştirme (Combination of Two Algorithms)

Yaptığımız araştırmalar ve deneme eğitimlerimize göre YOLOv5l6 modelinin büyük nesneleri, YOLOv5m6 modelinin ise daha küçük nesneleri bulmada verimli olduğuna karar verilmiş ve YOLOv5l6 da araçlar, UAP ve UAİ alanlarını; YOLOv5m6 modelinde ise insanları algıladığımız bir algoritma oluşturulup nesnenin doğruluk oranı artırılmıştır.

4.3. UAP ve UAİ Veri Seti Genişletme

Elimizde bulunan UAP ve UAİ veri setlerinde eksiklik olduğunu düşündüğümüz ve uçan aracın ineceği alanın güvenliğinin öneminden ötürü veri setimizi geliştirmek için örnek videodakiyle verilen benzer büyüklükte yaptırdığımız UAP ve UAİ alanlarını kendi droneumuzla çekimlerini yapıp veri setimize ekledik.

4.4. Ürettiğimiz Drone ile Veri Seti Oluşturma

Motor tercihimizi piyasadaki en iyi markalardan olan T-Motor'dan yana kullandık. Mevcut motorla 6s 25.2Volt 1300mAh pille çalışan 1.8kg itme gücü elde ediyoruz. Kontrol kartı drone için en önemli parça olup motor ve kumanda sinyallerini işleyip cevap veren bir kontrol ünitesidir. Mevcut sistemde Stm32f405 modeli kullanılmıştır. Gövde olarak karbon fiber malzemeden yapılmış Mark4 marka gövde tercih edilmiştir. Mevcut gövdeyle 5 inç pervane kullanılabilir. Çekimlerimiz için 4K 30FPS görüntü kaydı alabilen GoPro Hero 5 Black kamera kullanılmıştır.



Şekil 4.1 T-Motor



Şekil 4.2 Kontrol Kartı



Şekil 4.3 Mark4 Gövde



Şekil 4.4 GoPro Kamera



Şekil 4.5 1300mAh Batarya

4.5. Fotoğraf Filtreleme

Yarışma esnasında TEKNOFEST tarafından verilecek olan karlı, yağmurlu ve sisli hava görüntüleri için veri setimizde ki gündüz görüntülerinin bazılarını Adobe Photoshop ile efekt işlemleri gerçekleştirilmiştir [6].



Şekil 4.6 Adobe Photoshop ile Verilen Sisli Hava Efektli

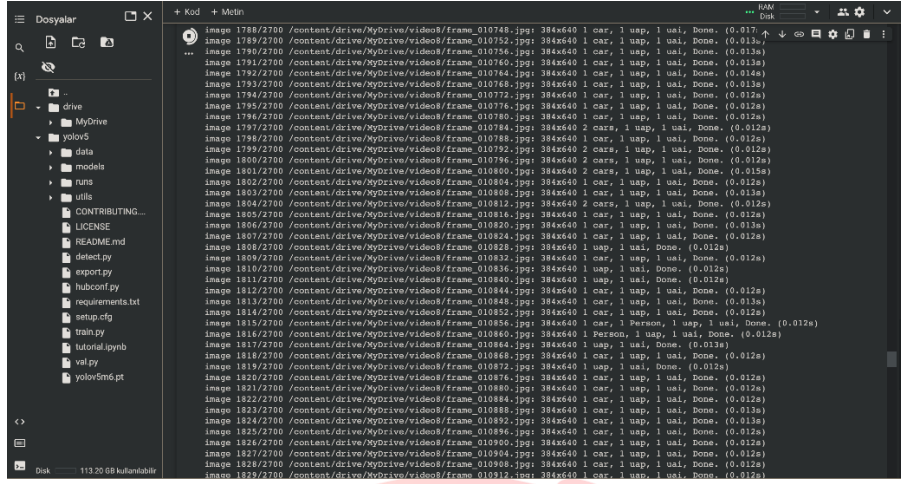
4.6. Deneme Eğitim Sonuçlarını Kullanma

Yaptığımız veri seti etiketleme işlemleri ardından deneme eğitimlerimize çıktı sonucu vermesi adına elimizde bulunan ve etiketlememiz gereken veri setlerini koyduk. Etiketlememiz gereken veriyi eğitimin verdiği çıktıdaki exp dosyasının içindeki labels klasöründen alarak elimizdeki framerle birleştirdik. Etiketlerde düzeltilmesi gereken yerler var ise düzelterek etiketlenmiş veri setimizi eğitim çıktılarından oluşturduk.

5. SONUÇLAR VE İNCELEME

Bir veya birden fazla algoritma sonuçlarının arasından oluşturduğumuz Combination of Two Algorithms algoritmasını kullanarak sistemlerin beraber çalışmasını sağladık. Ancak bu sistemde bile %100 doğruluk ve temiz bir çıktıya sahip olamıyoruz. Bu sorun devam ederse küçük nesneleri algılamada iyi sonuç veren YOLOv5m6 modelini tek başına kullanabiliriz.

Eğitim sırasında Google Colab kullanarak birçok deneme yapma şansımız oldu. Bu denemelerde de eğitim sonucunu kritik derecede etkileyen batch size, epoch, image size gibi optimizasyon ve hiper parametreleri kullanarak doğruluk oranımızı olabildiğince artırmaya çalıştık. Şu anlık en iyi çıktımızı 640x640 resim boyutunda, batch size 32 ve epoch değeri 150 iken aldık. Bu değerleri yarışma gününe kadar değiştirip optimum değerleri bulup kullanmayı planlıyoruz.



Şekil 5.1 Eğitim Süreci Google Colab Çıktıları

Ayrıca parametre araştırması yaparken YOLO mimarisinde bir kolaylık fark ettik. YOLO kullanıcılarına train sonrası detect edeceği video veya framelerin detect çıktısını txt olarak verebiliyor. Biz de bunu kullanarak ilk eğitimimizden sonra etiketlememiz gereken videoları önce detect edip yapay zekaya genel hatlarını etiketletiyoruz. Sonrasında frame içerisinde algılanmayan bir nesne varsa onu etiketleyip datasetimizin içerisine koyuyoruz. Yani sistemi özetlemek gerekirse yapay zeka bizim yerimize etiketlemenin bir çoğunu yapıyor. Bizde bu sayede hem zamandan kar etmiş oluyoruz hem de bu zamana kadar geliştirmiş olduğumuz sistemi kullanmış oluyoruz. Ayrıca bu sistemle daha hızlı dataset oluşturup sistemi kolayca güçlendirebiliyoruz. Kritik tasarım raporundan sonraki eğitimlerimizde yine dataset eksikliği hissederseniz oyun motorlarıyla verilerimizi güçlendirmeyi planlıyoruz.



6. KAYNAKÇA

- [1] <https://www.photoshopessentials.com/photo-effects/photoshop-weather-effects-rain/>
- [2] <https://github.com/ultralytics/yolov5/>
- [3] https://colab.research.google.com/?utm_source=scs-index/
- [4] <https://github.com/tzutalin/labelImg/>
- [5] <https://merveelifsarac.medium.com/cnn-r-cnn-fast-r-cnn-mask-r-cnn-c90a1a4d76fb/>
- [6] <https://www.mobilhanem.com/opencvde-filtreleme-islemlerine-giris/>

